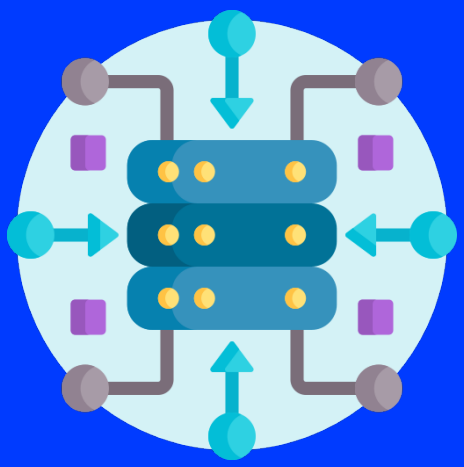
{EPITECH}

JOB AGGREGATOR_

< BUILD IT FOR REAL />



JOB AGGREGATOR

Build a real product, not just a school demo.

Your team will design and ship a complete job and internship aggregator platform that collects external offers, standardizes data, and helps users make better decisions.

The project must demonstrate your ability to integrate product thinking, fullstack engineering, deployment, automation, cybersecurity, and data/AI into one coherent solution.

Project Spirit

You are expected to demonstrate that you know how to:

- ✓ Build a working platform from end to end.
- ✓ Make clear product choices and justify them.
- ✓ Handle realistic technical constraints.
- ✓ Collaborate as one engineering team, not as isolated specialists.

⚠️ Do not split the project into disconnected specialty blocks. A feature is valid only if product, data, backend, frontend, security, and deployment decisions are aligned.

Partnership With WeLoveDevs

WeLoveDevs is a recognized tech hiring platform focused on connecting developers with great engineering teams and meaningful opportunities. For this project, WeLoveDevs agreed to open API access specifically for Epitech students so you can work with real-world market data, not synthetic classroom samples. Through this partnership, you are building on top of a real ecosystem used by real candidates and companies. ✨

> Take time to explore WeLoveDevs as a platform too: it can help you prepare and accelerate your search for internships, apprenticeships, and full-time jobs.

You have access to real-world job data through the WeLoveDevs API.

- ✓ API documentation (OpenAPI 3.0): `https://epi-api.welovedevs.com/`
- ✓ API key request form: `https://epi-api.welovedevs.com/ask-for-api-key/`
- ✓ Key requests are restricted to `@epitech.eu` email addresses.
- ✓ Rate limit: `1 request/second/student`
- ✓ A support chatbox is available in the API documentation.

> Using WeLoveDevs as a data source is mandatory in this project. You may add other sources, but WeLoveDevs integration is required.

Core Product Requirements

Your platform must include the following minimum scope.

1. Frontend

Provide a responsive and accessible interface with at least:

- ✓ Registration and authentication;
- ✓ User dashboard displaying real ingested data, with at least one output from the data feature and one output from the AI feature visible without additional navigation;
- ✓ Offer search and filtering;
- ✓ Offer details page displaying at minimum: title, company, location, contract type, date, description, and salary when available;
- ✓ Admin interface for moderation/management actions.

Your interface must meet **WCAG 2.1 Level AA** on at least three core flows: registration, offer search, and offer details. Document the result of at least one automated accessibility audit (Lighthouse, axe, or tota11y) in your repository.

2. Backend API

Design and implement a REST API that supports your product flows.

- ✓ Return proper HTTP status codes;
- ✓ Validate inputs;
- ✓ Handle errors cleanly;
- ✓ Protect sensitive routes with authentication and authorization.

3. Database

Use a relational or document database with a documented schema/model.

- ✓ Data model must support users, offers, and roles;
- ✓ Data integrity constraints must be explicit;
- ✓ Persistence must survive container restart.

4. Roles and Access Control

Role management is required.

- ✓ At minimum: `user` and `admin` roles;
- ✓ User role: can browse and use personal features;
- ✓ Admin role: can access administration features (for example, offer moderation and user management);
- ✓ Access control must be enforced server-side.

5. Containerization and Run Experience

You must use Docker and Docker Compose.

- ✓ At least 3 services: frontend, backend, database;
- ✓ Services must communicate correctly;
- ✓ Secrets/configuration must not be hardcoded;
- ✓ One command must start the full stack locally.

Data and AI Requirements

Data Collection

Your platform must collect external offers and allow manual trigger on demand.

- ✓ WeLoveDevs must be integrated as a required source;
- ✓ A second source is optional, but if used, explain the added value;
- ✓ Data ingestion must include normalization/pre-processing before storage.

Offer records should include at least:

- ✓ title;
- ✓ description;
- ✓ company/poster;
- ✓ location;
- ✓ date/duration;
- ✓ salary/remuneration when available.

Data Feature (Required)

Implement one data-centric feature that materially improves exploration or decision-making, such as:

- ✓ advanced filtering/sorting/comparison;
- ✓ dashboard visualizations;
- ✓ trend or salary distribution analytics;
- ✓ relevance scoring.

AI Feature (Required)

Implement one lightweight AI feature that solves a concrete task, such as:

- ✓ classification;
- ✓ recommendation;
- ✓ keyword/attribute extraction;
- ✓ duplicate detection;
- ✓ summarization;
- ✓ translation.

⚠️ You are not allowed to outsource the AI feature to a model API you do not control.
Heavy or resource-intensive models are forbidden: your model must fit within **500 MB** on disk, produce a result in under **5 seconds** per offer, and must not require fine-tuning at container startup.

Innovation Justification (Required)

For your dashboard and advanced features, you must justify:

- ✓ the user problem addressed;
- ✓ your feature hypothesis;
- ✓ why this approach is relevant;
- ✓ how success will be measured.

Cybersecurity Requirements

Secure both the application and the data.

You must address at least:

- ✓ secret management;
- ✓ injection protection (at least one type demonstrated);
- ✓ brute-force mitigation;
- ✓ route/session/token security;
- ✓ secure authentication flow;
- ✓ secure data handling in transit and at rest where applicable.

> A custom error middleware and centralized validation layer usually make both security and maintainability stronger.

CI and Engineering Workflow

You must use **GitHub Actions** as your CI system.

Your pipeline must run automatically on pull requests and pushes to main and include at least:

- ✓ build checks;
- ✓ lint/quality checks;
- ✓ automated tests covering at minimum two backend API routes and one data normalization function.

Repository expectations:

- ✓ frequent, descriptive commits;
- ✓ clean structure;
- ✓ no unnecessary large files;
- ✓ clear setup and run documentation.

Product, Business, and Team Management

Before and during implementation, produce concrete product evidence.

Market and Product Discovery

- ✓ Analyze trends, actors, and competing solutions;
- ✓ Identify user needs;
- ✓ Define your value proposition;
- ✓ Produce wireframes/mockups for key views.

Project Management

- ✓ Break work into tasks;
- ✓ Assign owners;
- ✓ Define timeline and priorities;
- ✓ Track progress with a visible tool.

Documentation of Choices

Your project documentation must explain decisions, not only list implemented features.

Create a concise decision-oriented document (or sectioned set of documents) that covers at least:

- ✓ Product choices: target users, prioritized pain points, and expected user value;
- ✓ Dashboard choices: why each key widget/feature exists and which user decision it supports;
- ✓ Architecture choices: selected stack, service boundaries, and rejected alternatives;
- ✓ Data choices: WeLoveDevs integration strategy, ingestion flow, rate-limit handling (`1 request/second/student`), and optional extra-source rationale;
- ✓ AI choices: selected method/model, why it is lightweight, expected limitations, and evaluation approach;
- ✓ Security choices: main threats considered and implemented controls;
- ✓ CI choices: what is automated, why those checks were selected, and how the pipeline protects quality.

Minimum format expectations:

- ✓ Each major choice must include `Why`, `How`, and `Trade-off`;
- ✓ Claims must be linked to evidence (code path, workflow file, screenshot, metric, or test result);
- ✓ At least one rejected alternative must be documented for architecture, data, or AI.

Architecture Decision Records (ADR) are a lightweight industry-standard format for this kind of documentation. A one-page ADR per major choice is sufficient. See adr.github.io for reference templates.

Deliverables

You must deliver:

- ✓ a functional local platform;
- ✓ a responsive frontend and usable dashboard;
- ✓ backend API code + documentation;
- ✓ documented database schema/model;
- ✓ data collection system (manual trigger required);
- ✓ one data feature + one AI feature integrated;
- ✓ security measures documentation (`Why / How / What`);
- ✓ decision documentation for product, architecture, data, AI, security, and CI choices (`Why / How / Trade-off` + evidence);
- ✓ product documentation (market scan, feature rationale, wireframes);
- ✓ project management evidence (`What / Who / When`);
- ✓ CI workflow files in repository.

BONUS

These are not required for validation, but are examples that can strengthen your final demonstration:

- ✓ online deployment;
- ✓ chatbot assistant;
- ✓ action/event logging and monitoring;
- ✓ anomaly alerting;
- ✓ advanced analytics and observability.

Final Presentation

Plan for approximately **20 minutes of live demonstration** followed by a **10-minute Q&A** with the jury.

You must present:

- ✓ a live demo of the running product covering at least two distinct user flows;
- ✓ a walkthrough of at least one technical decision in the code or architecture;
- ✓ evidence of collaboration across the team;
- ✓ feature decisions with measurable or demonstrated impact.

Clear communication is part of engineering. Show decisions, not only outcomes.

v 1

{EPITECH}